# Hair Salon Booking System

Student Name: Daniela Zelaya

Course: Data Base CS 1103

Professor: Jong-Kyou Kim

Date: 2026-04-13

## Abstract

This project presents the design and implementation of a Hair Salon Booking System Database. The purpose of the system is to manage the main operations of a salon in an organized and efficient way. The database stores information about customers, stylists, services, appointments, and payments. The project includes an Entity Relationship (ER) diagram, Data Definition Language (DDL) statements, sample data insertion, and a Java program connected to the database through JDBC. The system demonstrates how a relational database can be used to support appointment scheduling and payment tracking in a hair salon environment.

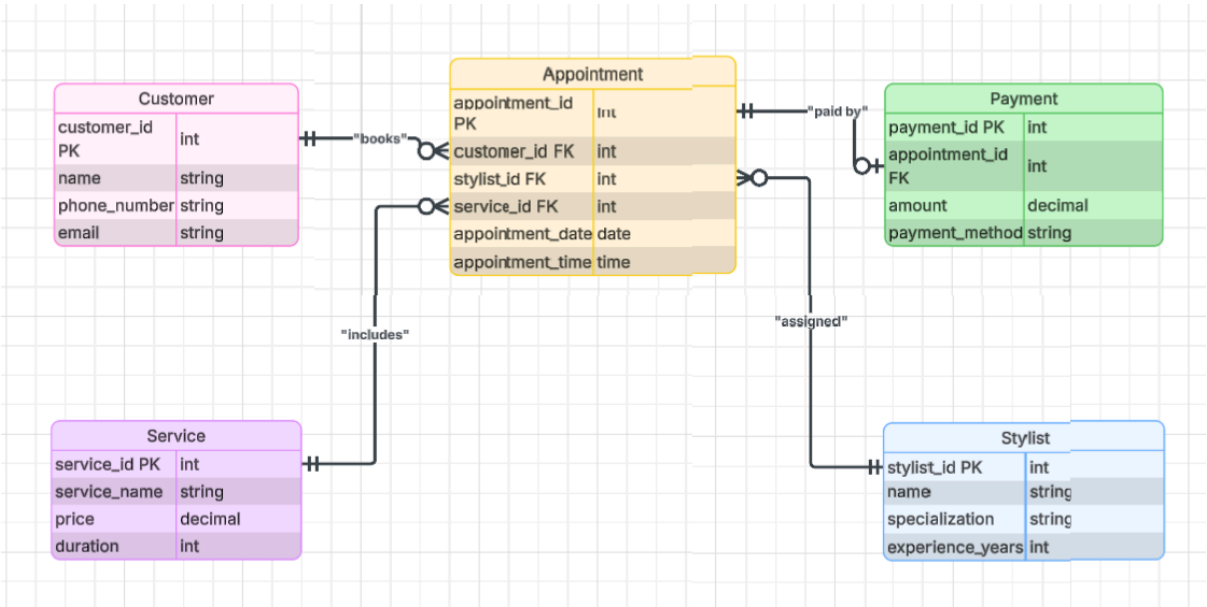
## Introduction

A hair salon handles many types of information every day, including customer details, stylist assignments, available services, appointment schedules, and payment records. Managing this information manually can lead to confusion, errors, and poor organization. A database system provides a structured way to store, retrieve, and manage salon data.

The purpose of this project is to design and implement a Hair Salon Booking System Database that can support the main activities of a salon. The system is designed around five main entities: Customer, Stylist, Service, Appointment, and Payment. These entities represent the most important parts of the booking and payment process.

This project includes the design of the database using an ER diagram, the implementation of the database structure using SQL DDL statements, and the use of Java with JDBC to connect to the SQLite database and perform operations such as inserting, selecting, updating, and deleting records. The project demonstrates how database concepts can be applied to a real-world scenario in a practical and organized way.

# ER Diagram


Customer
customer_id PK
int
name
string
phone_number
string
email
string
"books"
Appointment
appointment_id PK
int
customer_id FK
int
stylist_id FK
int
service_id FK
int
appointment_date
date
appointment_time
time
"paid by"
Payment
payment_id PK
int
appointment_id FK
int
amount
decimal
payment_method
string
"includes"
"assigned"
Service
service_id PK
int
service_name
string
price
decimal
duration
int
Stylist
stylist_id PK
int
name
string
specialization
string
experience_years
int

## DDL Statements:

```
CREATE TABLE Customer (
   customer_id INTEGER PRIMARY KEY,
   name TEXT,
   phone_number TEXT,
   email TEXT
);

CREATE TABLE Stylist (
   stylist_id INTEGER PRIMARY KEY,
   name TEXT,
   specialization TEXT,
   experience_years INTEGER
);

CREATE TABLE Service (
   service_id INTEGER PRIMARY KEY,
   service_name TEXT,
   price REAL,
   duration INTEGER
);

CREATE TABLE Appointment (
   appointment_id INTEGER PRIMARY KEY,
   customer_id INTEGER,
   stylist_id INTEGER,
   service_id INTEGER,
   appointment_date TEXT,
   appointment_time TEXT
);

CREATE TABLE Payment (
   payment_id INTEGER PRIMARY KEY,
   appointment_id INTEGER,
   amount REAL,
   payment_method TEXT
);
```

## Explanation of each table:

- **Customer:** This table basically stores information such as name, phone number, and email.

- **Stylist:** This table stores information about the stylists, including their specialization and years of experience.

- **Service:** This table stores services offered by the salon, including price and duration.

- **Appointment:** This table stores booking information and connects customers, stylists, and services for a specific date and time.

- **Payment:** This table stores payment information related to appointments, including amount and payment method.

## Sample Statement:

The following sample INSERT statements were used to add data into the database tables.These sample records were used to test the database and confirm that the system could successfully store related information across multiple tables.

```
INSERT INTO Customer VALUES (99, 'Alice Brown', '506-123-4567', 'alice@email.com');

INSERT INTO Stylist VALUES (1, 'Emma Lee', 'Hair Coloring', 5);

INSERT INTO Service VALUES (1, 'Haircut', 30.0, 45);

INSERT INTO Appointment VALUES (3, 99, 1, 1, '10:00', '2026-03-29');

INSERT INTO Payment VALUES (1, 3, 30.0, 'Cash');
```

## JDBC & Java Implementation

For this project, SQLite was used as the database system and BlueJ was used to write and test the Java code. JDBC was used to connect Java to the SQLite database. This method was chosen because it allowed the project to move beyond the design stage and actually test how the database would work in practice through a program.

By using Java and JDBC, it was possible to perform important database operations such as inserting, selecting, updating, and deleting data. This made the project more practical, since it showed how the salon database could be used in a real application and not only stored as a database design.

BlueJ was used as the compiler and programming environment because it made the process of testing the code easier and more organized. It allowed the database connection and SQL operations to be checked step by step. This helped confirm that the system was working correctly.

Overall, this implementation shows how Java can be connected to SQLite through JDBC to create a working database application for managing salon customers, appointments, services, stylists, and payments.

## System Functionality

The Hair Salon Booking System Database supports the following functions:

- **Customer Management**

  The system stores customer information, including customer ID, name, phone number, and email. This allows the salon to keep an organized record of its clients.

- **Stylist Management**

  The system stores stylist information, including stylist ID, name, specialization, and years of experience. This helps the salon track which stylists are available and what type of service they specialize in.

- **Service Management**

  The system stores information about salon services, including service name, price, and duration. This makes it possible to assign the correct service to an appointment.

- **Appointment Scheduling**

  The Appointment table acts as the central part of the system. It connects the customer, stylist, and service for a specific date and time. This allows the salon to manage bookings in an organized way.

- **Payment Tracking**

  The Payment table stores the payment information for completed appointments. This includes payment ID, appointment ID, amount, and payment method.

## SQL and JDBC Operations Performed

The project demonstrated the following database operations:

- **Insert Operation**

  Records were inserted into all five tables: Customer, Stylist, Service, Appointment, and Payment.

- **Select Operation**

  SELECT queries were used to display customer, appointment, and payment information. A JOIN query was used to display appointment details together with customer name, stylist name, and service name.

- **Update Operation**

  An UPDATE query was used to modify customer information, such as changing a phone number.

- **Delete Operation**

  A DELETE query was used to remove a customer record from the database.

These operations basically show that the system supports the basic functionality required for a relational database application.

## Output Examples

The following sample outputs were produced by the Java program during testing.

**Appointment Output:**

Appointment List:
1 | Alice Brown | Emma Lee | Haircut | 2026-03-29 | 10:00
2 | Alice Brown | Emma Lee | Haircut | 2026-03-29 | 10:00
3 | Alice Brown | Emma Lee | Haircut | 2026-03-29 | 10:00

**Payment Output:**

Payment List:
1 | 3 | 30.0 | Cash

So basically these outputs confirm that the tables are connected correctly and that the data base is functioning as imntended.

## Conclusion

In conclusion, this project successfully developed a Hair Salon Booking System Database for managing customers, stylists, services, appointments, and payments. The project showed how an ER diagram can be turned into SQL tables and then connected to Java using JDBC.

By testing operations such as insert, select, update, and delete, the project became more than just a database design and showed how the system could work in practice. Overall, this project helped me better understand how databases can be designed, implemented, and used in a real-world situation.

# Appendice:

## Appendix 1. Java Source Code:

This appendix contains the Java source code used to connect the Hair Salon Booking System Database to SQLite through JDBC. The code was used to test the database connection and perform key operations such as inserting appointments, displaying appointments, inserting payments, and displaying payment records.

### A.1 Database Connection Test

This proves that i connected Java to SqLite (I forgot to save my changes separately or took ss,  i kept updating in the same class in Java but this is the code that i used)

```
import java.sql.Connection;
import java.sql.DriverManager;

public class SalonDBTest
{
    public static void main(String[] args)
    {
        String url = "jdbc:sqlite:C:/Users/danie/OneDrive/salon.db";

        try
        {
            Connection conn = DriverManager.getConnection(url);
            System.out.println("Connected successfully!");
            conn.close();
        }
        catch (Exception e)
        {
            System.out.println("Connection failed.");
            e.printStackTrace();
        }
    }
}
```

## A.2 Insert Appointments

This is one of the most important i guess, because it shows the system actually booking an appointment.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.ResultSet;
import java.sql.PreparedStatement;

public class SalonDBTest
{
    public static void main(String[] args)
    {
        String url = "jdbc:sqlite:C:/Users/danie/OneDrive/salon.db";

        try
        {
            Connection conn = DriverManager.getConnection(url);
            Statement stmt = conn.createStatement();

            ResultSet rsCustomer = stmt.executeQuery("SELECT customer_id FROM
Customer LIMIT 1");
            if (!rsCustomer.next())
            {
                System.out.println("No customer found.");
                rsCustomer.close();
                stmt.close();
                conn.close();
                return;
            }
            int customerId = rsCustomer.getInt("customer_id");
            rsCustomer.close();

            ResultSet rsStylist = stmt.executeQuery("SELECT stylist_id FROM Stylist
LIMIT 1");
            if (!rsStylist.next())
            {


               System.out.println("No stylist found.");
               rsStylist.close();
               stmt.close();
               conn.close();
               return;
            }
            int stylistId = rsStylist.getInt("stylist_id");
            rsStylist.close();

            ResultSet rsService = stmt.executeQuery("SELECT service_id FROM Service
LIMIT 1");
            if (!rsService.next())
            {
                System.out.println("No service found.");
                rsService.close();
                stmt.close();
                conn.close();
                return;
            }
            int serviceId = rsService.getInt("service_id");
            rsService.close();

            ResultSet rsApp = stmt.executeQuery("SELECT IFNULL(MAX(appointment_id),
0) + 1 AS next_id FROM Appointment");
            int appointmentId = 1;
            if (rsApp.next())
            {
                appointmentId = rsApp.getInt("next_id");
            }
            rsApp.close();

            String sql = "INSERT INTO Appointment (appointment_id, customer_id,
stylist_id, service_id, appointment_date, appointment_time) VALUES (?, ?, ?, ?, ?,
?)";
```

```
            PreparedStatement pstmt = conn.prepareStatement(sql);

            pstmt.setInt(1, appointmentId);
            pstmt.setInt(2, customerId);
            pstmt.setInt(3, stylistId);
            pstmt.setInt(4, serviceId);
            pstmt.setString(5, "2026-03-29");
            pstmt.setString(6, "10:00");

            int rows = pstmt.executeUpdate();

            System.out.println("Appointment inserted successfully! Rows added: " +
rows);
            System.out.println("customer_id used: " + customerId);
            System.out.println("stylist_id used: " + stylistId);
            System.out.println("service_id used: " + serviceId);
            System.out.println("appointment_id used: " + appointmentId);

            pstmt.close();
            stmt.close();
            conn.close();
        }
        catch (Exception e)
        {
            System.out.println("Insert appointment failed.");
            e.printStackTrace();
        }
    }
}
```

## A.3 Display Appointments

This is also very important because it shows the relation between tables.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.ResultSet;

public class SalonDBTest
{
    public static void main(String[] args)
    {
        String url = "jdbc:sqlite:C:/Users/danie/OneDrive/salon.db";

        try
        {
            Connection conn = DriverManager.getConnection(url);
            Statement stmt = conn.createStatement();

            String sql = "SELECT a.appointment_id, " +
                         "c.name AS customer_name, " +
                         "s.name AS stylist_name, " +
                         "sv.service_name, " +
                         "a.appointment_date, " +
                         "a.appointment_time " +
                         "FROM Appointment a " +
                         "LEFT JOIN Customer c ON a.customer_id = c.customer_id " +
                         "LEFT JOIN Stylist s ON a.stylist_id = s.stylist_id " +
                         "LEFT JOIN Service sv ON a.service_id = sv.service_id";

            ResultSet rs = stmt.executeQuery(sql);

            System.out.println("Appointment List:");
            while (rs.next())
            {
                System.out.println(
                    rs.getInt("appointment_id") + " | " +
                    rs.getString("customer_name") + " | " +
                    rs.getString("stylist_name") + " | " +
                    rs.getString("service_name") + " | " +
                    rs.getString("appointment_date") + " | " +
                    rs.getString("appointment_time")
                );
            }

            rs.close();
            stmt.close();
            conn.close();
        }
        catch (Exception e)
        {
            System.out.println("Select appointments failed.");
            e.printStackTrace();
        }
    }
}
```

## A.4 Insert Payments

This helps show the project is basically complete.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.Statement;
import java.sql.ResultSet;

public class SalonDBTest
{
    public static void main(String[] args)
    {
        String url = "jdbc:sqlite:C:/Users/danie/OneDrive/salon.db";

        try
        {
            Connection conn = DriverManager.getConnection(url);
            Statement stmt = conn.createStatement();

            ResultSet rsPay = stmt.executeQuery("SELECT IFNULL(MAX(payment_id), 0) +
1 AS next_id FROM Payment");
            int paymentId = 1;
            if (rsPay.next())
            {
                paymentId = rsPay.getInt("next_id");
            }
            rsPay.close();

            ResultSet rsApp = stmt.executeQuery("SELECT appointment_id FROM
Appointment ORDER BY appointment_id DESC LIMIT 1");
            if (!rsApp.next())
            {
                System.out.println("No appointment found.");
                rsApp.close();
                stmt.close();
                conn.close();
                return;
            }
            int appointmentId = rsApp.getInt("appointment_id");
            rsApp.close();

            String sql = "INSERT INTO Payment (payment_id, appointment_id, amount,
 payment_method) VALUES (?, ?, ?, ?)";
            PreparedStatement pstmt = conn.prepareStatement(sql);

            pstmt.setInt(1, paymentId);
            pstmt.setInt(2, appointmentId);
            pstmt.setDouble(3, 30.0);
            pstmt.setString(4, "Cash");

            int rows = pstmt.executeUpdate();

            System.out.println("Payment inserted successfully! Rows added: " + rows);
            System.out.println("payment_id used: " + paymentId);
            System.out.println("appointment_id used: " + appointmentId);

            pstmt.close();
            stmt.close();
            conn.close();
        }
        catch (Exception e)
        {
            System.out.println("Insert payment failed.");
            e.printStackTrace();
        }
    }
}
```

## A.5 Display Payments

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.ResultSet;

public class SalonDBTest
{
    public static void main(String[] args)
    {
        String url = "jdbc:sqlite:C:/Users/danie/OneDrive/salon.db";

        try
        {
            Connection conn = DriverManager.getConnection(url);
            Statement stmt = conn.createStatement();

            ResultSet rs = stmt.executeQuery("SELECT * FROM Payment");

            System.out.println("Payment List:");
            while (rs.next())
            {
                System.out.println(
                    rs.getInt("payment_id") + " | " +
                    rs.getInt("appointment_id") + " | " +
                    rs.getDouble("amount") + " | " +
                    rs.getString("payment_method")
                );
            }

            rs.close();
            stmt.close();
            conn.close();
        }
        catch (Exception e)
        {
            System.out.println("Select payments failed.");
            e.printStackTrace();
        }
    }
}
```

## Appendix 2 Sample Outputs :

### B.1 Appointment Output

This appendix contains sample outputs from java program during testing.

```
Appointment List:
1 | Alice Brown | Emma Lee | Haircut | 2026-03-29 | 10:00
2 | Alice Brown | Emma Lee | Haircut | 2026-03-29 | 10:00
3 | Alice Brown | Emma Lee | Haircut | 2026-03-29 | 10:00
```

### B.2 Payment Output:

```
Payment List:
1 | 3 | 30.0 | Cash
```